Borealis

設定ファイル (`borealis.yml`) の対応表

キー・既定値・選べる値・効かないキー

高橋裕介

2026 年 10 月 20 日 (Borealis version 1.7.0)

目次

1 **この表の読み方** 2

2 **設定ファイルの書き方** 2

3 adapter と optimizer 3

4 objectives 4

5 parameter_optimized 4

6 PSO 5

7 ABC 6

8 GA 7

9 Bayesian_optimization 8

10 reference 9

11 tacode 10

12 simple_function 10

13 example_externalcode 11

14 heatcond 11

15 cage 12

16 shapeoptimizer 12

17 user 14

18 **読まれないキーの一覧** 14

1 この表の読み方

この文書は、Borealis が読む設定ファイル `borealis.yml` に書けるキーを、セクションごとにすべて並べたものである。**2026 年 10 月 20 日の時点のソース（version 1.7.0）**に合わせてある。キーは `src/` の Python が `config[...]` と `config.get(...)` で読む箇所をすべて拾って作った（`previous/` と、外してある `optimizer_ga` は除く）。

表の列は次のとおりである。

- **キー**：セクションからの道筋。リストの要素は `[]` で書いた（`boundary[].component[].bound_min` など）
- **既定**：「必須」は書かないと `KeyError` で止まるもの。「必須*」は、ある条件のときだけ必須のもの（条件は説明に書いた）。値が書いてあるものは省略でき、そのときの値である
- **選べる値**：語で書くキーは語の一覧、数値のキーは意味と単位
- **説明・注意**：何に効くか、どの組み合わせで意味を持つか、気をつける点

既知の問題は、リポジトリの外に置いた開発の記録の番号で示す（「既知の問題 4」など）。

2 設定ファイルの書き方

2.1 枠は YAML

設定ファイルは YAML で書く。最上位のキーはおおむね「モジュールの名前 = セクションの名前」になっている。

```
adapter:
  kind_adapter: simple_function
optimizer:
  kind_optimizer: PSO
objectives:
  num_objectives: 1
parameter_optimized:
  boundary:
    - name: 'simple_function'
      component:
        - {type: x1, bound_min: -10, bound_max: 10}
        - {type: x2, bound_min: -10, bound_max: 10}
  flag_penalty: False
PSO:
  num_optiter: 100
  num_particle: 20
  ...
simple_function:
  ...
```

- **既定では、作業ディレクトリの `borealis.yml` を読む。** 別の場所のファイルは `python3 src/borealis.py -file path/to/borealis.yml` で渡す。相対のパスは、すべて作業ディレクトリから解く
- **選んでいないセクションは読まない。** `kind_optimizer: PSO` のとき、`ABC` や `Bayesian_optimization` のセクションは書いてあっても無視される。雛形の yml に他のセクションが並んでい

るのはこのため

- **知らないキーは、起動時に警告が出る**（Warning in borealis.yml : PSO.num_particles: unknown key, not read など。src/config_check/）。止めはしないので、**警告を読むこと**。読まれるが効かないキー、古い綴りのキーも警告する（既知の問題 4）
- **起動すると消されるディレクトリがある。 <オプティマイザ>**.result_dir と **<アダプタ>**.work_dir（多くのアダプタでは **<アダプタ>**.result_dir も）は、起動したときに中身ごと消して作り直す。残したい結果のあるディレクトリを指さないこと

2.2 テンプレートの場所の指定

外部コードを呼ぶアダプタは、テンプレートのケースの場所とリファレンスの場所を、どちらも次の 2 つのキーで指定する。

- directory_path_specify：manual なら manual_path を使う。default・auto なら src/orbital/ からの既定の場所を使う。それ以外の語は止まる
- manual_path：default のときも、書いていないと KeyError で止まる

default・auto は、1.7.0 で直るまでパスの連結に区切りが抜けていて使えなかった（既知の問題 26）。既定の場所は雛形の配置に合わせたものなので、ふつうは manual で書く。

2.3 外部コードの実行

cmd_code（example_externalcode だけは cmd_externalcode）は、シェルを通さずに 1 つの実行ファイルとして起動する。**引数は付けられない。** 引数が要るなら、それを書いたシェルスクリプトを置いて、その名前を書く。**終了ステータスが 0 でないか、結果のファイルが書かれなければ、評価の失敗**として扱い、目的関数の値は parameter_optimized.value_nonfinite になる。テンプレートから写した古い結果のファイルは、起動の前に消す。実行ファイルは、ケースのディレクトリ（テンプレートを写したもの）の中で起動するので、./run_code.sh のように書くのがふつうである。実行の権限を付けておくこと。

3 adapter と optimizer

キー	既定	選べる値	説明・注意
adapter.kind_adapter	必須	tacode・simple_function・example_externalcode・heatcond・cage・shapeoptimizer・user	目的関数を与えるアダプタ。src/adapter_**<名前>**/ のクラスを読む（simple_function と example_externalcode は src/adapter_template/、user は src/hidden_adapter_user/）。一覧に無い語は案内を出して止まる
optimizer.kind_optimizer	必須	PSO・ABC・GA・Bayesian_optimization	最適化の手法
optimizer.seed	無し（毎回変わる）	整数	乱数の種。MPI では各ランクを seed + rank で初期化する。逐次の実行では、同じ種なら同じ答えになる

4 objectives

キー	既定	選べる値	説明・注意
objectives.num_objectives	1（PSO・ABC・GA）／必須（simple_function）	整数	目的関数の数。2 以上で PSO は非劣解のアーカイブ、GA は NSGA-II、ABC は支配関係と外部アーカイブによる多目的になる。出力の Error_k・Objective_k の列の数もこれで決まる。**ベイズ最適化は単目的だけ**

5 parameter_optimized

5.1 最適化するパラメータ

キー	既定	選べる値	説明・注意
parameter_optimized.boundary	必須	リスト	探索する範囲。各要素の component を順に並べたものが、パラメータのベクトルになる。simple_function は読まない。**user はここでなく Bayesian_optimization.boundary を読む**
boundary[].name	必須	文字列	制御ファイルの中で書き換える変数を探す手がかり。**アダプタごとに使い方が違う**（下の表）。ベイズ最適化は表示に使うだけ
boundary[].component	必須	リスト	その変数の成分（次元）。shapeoptimizer では、すべての要素で成分の数がそろっていないと止まる
boundary[].component[].type	必須	文字列	成分の名前。出力ファイルの列の名前になる
boundary[].component[].bound_min・bound_max	必須	数値	探索の下限と上限。PSO・ABC・ベイズ最適化は初期値をこの中から取る。ABC は候補をこの中に切り詰める。**PSO は切り詰めない**（既知の問題 21）。ペナルティの判定にも使う

boundary[].name の使い方は、アダプタごとに次のように違う。

アダプタ	探す文字列	書き換える場所
tacode（code_system: python3）・example_externalcode・user	name の最後の . より後ろ	見つけた行の次の N 行の 2 語目（YAML のリストの要素）
tacode（code_system: fortran）・heatcond	name 全体	見つけた行の、先頭から N 個の語（カンマ区切りで書く）
cage	name の最後の . より**前**	見つけた行の次の N 行の 2 語目
shapeoptimizer	使わない	boundary の要素の数をマーカーの数とする

5.2 初期値と、範囲の外のペナルティ

キー	既定	選べる値	説明・注意
parameter_optimized.boundary_initial	無し（boundary を使う）	bound_min・bound_max	PSO の粒子の初期位置を取る範囲。**すべての次元に同じ 1 つの範囲を使う**
parameter_optimized.velocity_initial	無し（[-1, 1]）	velocity_min・velocity_max	PSO の初速を取る範囲。すべての次元に同じ範囲。書かないと、パラメータの尺度によらず [-1, 1] になる
parameter_optimized.flag_penalty	必須（外部コードのアダプタ）	True・False	パラメータが bound_min〜bound_max の外なら、誤差に penalty_value を足す。simple_function・user は読まない
parameter_optimized.penalty_value	必須*（flag_penalty が真）	数値（shapeoptimizer は目的ごとのリストも可）	足す値。**違反の量によらない定数**（既知の問題 13）
parameter_optimized.value_nonfinite	1e30	数値	目的関数の値が NaN・無限大のとき、または外部コードが失敗したときに使う値（PSO・ABC）。警告が出る
parameter_optimized.flag_with_timeerror	False	True・False	tacode だけ。特定の高度を通る時刻の誤差も評価する形に切り替える。**真にすると tacode.time_start・time_end・target_time の単位が日から秒に変わる**
parameter_optimized.set_altitude・set_count_pass・weight_time	必須*（上が真）	高度 [km] のリスト（先頭だけ使う）・整数・数値	何回目にその高度を通る時刻を見るか、時刻の誤差の重み。weight_time **を掛けた時刻の誤差は、返す誤差に入っていない**（既知の問題 27）

6 PSO

src/optimizer_pso/optimizer_pso.py。アルゴリズムはレポートの「粒子群最適化」の節を見ること。

キー	既定	選べる値	説明・注意
PSO.num_particle	必須	整数	粒子の数。MPI では均等に分ける（np.array_split）。プロセス数より少ないと止まる
PSO.num_optiter	必須	整数	反復（ステップ）の最大の数
PSO.inertia	必須*	数値	慣性の重み w。*inertia_schedule: constant（既定）のとき必須で、ほかでは読まない
PSO.cognitive_coef	必須*	数値	個人の最良へ向かう係数 c1。*coefficient_schedule: constant（既定）のとき必須で、ほかでは読まない
PSO.social_coef	必須*	数値	リーダーへ向かう係数 c2。*同上
PSO.inertia_schedule	constant	constant・linear・adaptive	慣性の重みの決め方。constant は inertia で一定（以前と同じ答え）。linear は inertia_max から inertia_min へステップとともに線形に下げる。adaptive は、そのステップで個人の最良を更新した粒子の割合 S から w = inertia_min + (inertia_max − inertia_min)·S（停滞すると小さくなる）。進み具合は num_optiter を基準にする。局所解の多い問題では adaptive と c1 = c2 = 1.5 を先に試すとよい（レポートの「PSO の重み係数」）

キー	既定	選べる値	説明・注意
PSO.inertia_max・PSO.inertia_min	0.9・0.4	数値	linear・adaptive での w の上限と下限
PSO.coefficient_schedule	constant	constant・linear	加速係数の決め方。linear は c1 を cognitive_coef_initial から cognitive_coef_final へ、c2 を social_coef_initial から social_coef_final へ線形に変える（TVAC）
PSO.cognitive_coef_initial・cognitive_coef_final	2.5・0.5	数値	coefficient_schedule: linear での c1 の始めと終わり
PSO.social_coef_initial・social_coef_final	0.5・2.5	数値	coefficient_schedule: linear での c2 の始めと終わり
PSO.maximize	必須	True・False、または目的ごとのリスト（要素の数は num_objectives）	最大化なら符号 − 1 を目的関数に渡す。**符号を掛けるのはアダプタの側**で、対応しているのは shapeoptimizer・simple_function。**ほかのアダプタで真にすると止まる**（誤差を返すので最大化に意味が無い。既知の問題 1）
PSO.windowsize_residual	20	整数	収束判定で、何ステップ前の個人の最良と比べるか
PSO.tolerance_residual	1e-5	数値	個人の最良の相対変化の平均がこれ以下で収束とする
PSO.windowsize_archive	5	整数	多目的だけ。何ステップ前のアーカイブの重心と比べるか
PSO.tolerance_archive	1.0	数値	多目的だけ。アーカイブの重心の移動がこれ以下なら停滞とする。多目的の収束は「残差」かつ「停滞」
PSO.num_parallel	1	整数	1 プロセスの中で同時に評価する粒子の数（スレッド）。2 以上は、アダプタが並列評価に対応しているときだけ（いまは shapeoptimizer・simple_function。対応していなければ止まる）
PSO.result_dir	必須	文字列	出力のディレクトリ。**起動時に消して作り直す**
PSO.filename_output	必須	文字列	全粒子の位置・速度・誤差の履歴（Tecplot）
PSO.filename_global	必須	文字列	各ステップのアーカイブ（単目的では、そのステップの最良）の履歴
PSO.kind_residual_computation	relative_change	文字列	**表示するだけで効かない**（既知の問題 4）
PSO.tolerance	—	—	**読まれない**。古い名前。tolerance_residual を使う
PSO.times_spte_pso	—	—	**読まれない**

7 ABC

src/optimizer_abc/optimizer_abc.py。num_objectives が 2 以上なら多目的（支配関係で採否を決め、非劣解を外部のアーカイブに集める。レポートの「人工蜂コロニー」の節）。

キー	既定	選べる値	説明・注意
ABC.num_employ_bees	必須	整数	雇用蜂の数（= 食料源の数）。MPI では np.array_split で均等に分ける
ABC.num_onlook_bees	必須	整数	追従蜂の数。古い名前 num_onlook_bee も読む（警告が出る）
ABC.visit_limit	必須	整数	食料源を捨てるまでの、改善しなかった回数の上限（これを超えたら偵察蜂が作り直す。最良の食料源も対象）。古い綴り vist_limit も読む（警告が出る）
ABC.num_optiter	必須	整数	反復の最大の数
ABC.tolerance	必須	数値	単目的だけ（多目的では読むが判定に使わない）。windowsize_residual ステップ前の解からの相対変化の平均がこれ以下で打ち切る。改善の無いステップでは 0 になるので、早く止まりすぎるなら windowsize_residual を大きくする（既知の問題 5）
ABC.windowsize_residual	1	整数	収束判定で比べる相手（何ステップ前の解か）。最初のこのステップ数は判定しない
ABC.num_parallel	1	整数	1 プロセスの中で同時に評価する候補の数（スレッド）。対応するアダプタだけ。2 以上では、雇用蜂の候補を段の始まりの食料源から作るので、逐次とは答えが変わる（並列の数にはよらない）。初期化・追従蜂・偵察蜂は逐次と同じ
ABC.maximize	False	True・False、または目的ごとのリスト（要素の数は num_objectives）	PSO と同じく、符号を掛けるのはアダプタの側。対応しないアダプタでは止まる
ABC.num_archive	100	整数	多目的だけ。アーカイブの大きさの上限。超えたら混雑距離の最も小さい点から捨てる
ABC.windowsize_archive	5	整数	多目的だけ。何ステップ前のアーカイブの重心と比べるか
ABC.tolerance_archive	0	数値	多目的だけ。アーカイブの重心の移動がこれ以下なら止める。既定の 0 では、アーカイブが windowsize_archive ステップのあいだ変わらなかったときだけ止まる。**PSO の同じ名前のキーとは既定が違う**（PSO は 1.0）
ABC.result_dir・filename_output・filename_global	必須	文字列	PSO と同じ。filename_global には、単目的では各ステップの時点での**それまでの最良**を書く（1.7.0 から。既知の問題 16）。多目的では各ステップのアーカイブ（全ステップの評価から作ったもの）を PSO・GA と同じ形で書く

8 GA

src/optimizer_ga/optimizer_ga.py。実数値の GA を NSGA-II の世代交代で回す（1.7.0 で実装）。単目的では最良の個体が必ず残るエリート保存の GA、2 目的以上では NSGA-II になる。1 世代に評価するのは子の num_population 個で、評価の総数は num_population × (num_generation + 1)（初期集団を含む）。

キー	既定	選べる値	説明・注意
GA.num_population	必須	4 以上の整数	集団の大きさ N。MPI では子の評価を均等に分ける
GA.num_generation	必須	整数	世代の最大の数
GA.maximize	False	True・False、または目的ごとのリスト	PSO と同じく、符号を掛けるのはアダプタの側。対応しないアダプタでは止まる
GA.tournament_size	2	整数	トーナメント選択で比べる個体の数
GA.crossover_rate	0.9	0〜1	親の組に SBX を行う確率（行わなければ親をそのまま写す）
GA.eta_crossover	15	数値	SBX の分布指数。大きいほど子は親の近くにできる
GA.mutation_rate	1/次元	0〜1	成分ごとの多項式突然変異の確率
GA.eta_mutation	20	数値	多項式突然変異の分布指数。大きいほど小さくずらす。最適解の近くで精度を詰めたいときは大きくする
GA.windowsize_residual	20	整数	収束判定で、何世代前の集団の平均と比べるか
GA.tolerance_residual	1e-5	数値	集団の値の平均の相対変化がこれ以下で止める
GA.num_parallel	1	整数	1 プロセスの中で同時に評価する子の数（スレッド）。PSO と同じく、対応するアダプタだけ
GA.result_dir・filename_output・filename_global	必須	文字列	出力。filename_output は各世代の集団（パラメータ・番号・値・非劣解の順位・残差。世代 0 は初期集団）、filename_global は各世代の第 1 フロント（単目的では最良）。result_dir は起動時に消して作り直す
GA.mutation_limit・GA.tolerance	—	—	**読まれない**（1.6 までの試作の GA のキー。警告が出る）

9 Bayesian_optimization

ガウス過程による最適化を、bayesian-optimization（3.x、https://github.com/bayesian-optimization/BayesianOptimization）で行う（実装は src/Bayesian_optimization/ にある）。1.6 までは GPyOpt を使っていた。キーの名前は 1.6 と同じで、GPyOpt にしか無かった選択肢は無くなった。最初に initial_design_numdata 点を一様乱数で評価し、そのあと num_optiter 回、獲得関数が最大になる点を評価する。乱数の種は optimizer.seed。単目的だけで、MPI での分担はしない。

キー	既定	選べる値	説明・注意
Bayesian_optimization.num_optiter	必須	整数	初期の点のあとに、獲得関数で選んで評価する回数。評価の総数は initial_design_numdata + num_optiter
Bayesian_optimization.initial_design_numdata	5	整数	最初に一様乱数で評価する点の数
Bayesian_optimization.acquisition_type	EI	EI・PI（MPI も可）・LCB（UCB も可）	獲得関数。期待改善量・改善確率・信頼上限（最小化では下限）。*_MCMC は無くなった（指定すると止まる）
Bayesian_optimization.acquisition_xi	0.01	数値	EI・PI の探索の度合い（大きいほど探索寄り）
Bayesian_optimization.acquisition_weight	2.0	数値	LCB（UCB）の探索の重み kappa

キー	既定	選べる値	説明・注意
Bayesian_optimization.normalize_Y	True	True・False	ガウス過程で目的関数の値を標準化するか
Bayesian_optimization.model_type	GP	GP	代理モデル。ガウス過程だけ（ほかの語は止まる）
Bayesian_optimization.maximize	False	True・False	最大化。符号はベイズ最適化の側で扱うので、**どのアダプタでも効く**
Bayesian_optimization.result_dir・filename_output	必須	文字列	出力（評価した順のパラメータ・誤差・番号）。result_dir は起動時に消して作り直す
Bayesian_optimization.flag_image_convergence・filename_image_convergence	False・convergence.png	True・False、文字列	収束の図（それまでの最良）を保存する
Bayesian_optimization.acquisition_optimizer_type・verbosity・flag_image_acquisition・filename_image_acquisition	—	—	**読まれない**（GPyOpt の選択肢。警告が出る）
Bayesian_optimization.initial_design_type・model_update_interval	—	—	**読まれない**
Bayesian_optimization.boundary	必須*（kind_adapter: user）	parameter_optimized.boundary と同じ形	user のアダプタだけがここを読む（古い名残）

10 reference

誤差を評価するときの基準のデータ。simple_function・shapeoptimizer は読まない。

キー	既定	選べる値	説明・注意
reference.directory_path_specify・manual_path	必須	manual（default は使えない）	基準のデータの場所（上の「テンプレートの場所の指定」）
reference.filename_input	必須	文字列	基準のデータのファイル。読み方はアダプタごとに違う。tacode は **<名前>_<var_y の各要素>.<拡張子>** の 3 つのファイル（列は時刻・平均・標準偏差、1 行目は見出し）。example_externalcode・user は x, y（2 行飛ばす）。heatcond は Tecplot（1 行目が見出し）。cage はカンマ区切り（0 行目が見出し）
reference.var_x	必須	文字列	x（時刻）の列の名前。**tacode は読むだけで使わない**
reference.var_y	必須	tacode はリスト、heatcond は文字列、cage はリスト	tacode はファイルの名前の後ろ（順序が意味を持つ。[0] 経度、[1] 緯度、[2] 高度）。heatcond は計算の T[K] と比べる列。cage は cage.var_y と順に比べる列

キー	既定	選べる値	説明・注意
reference.timeunit	必須（tacode）	day・hour・min・sec	基準のデータの時刻の列（秒）をこれで割って比べる。day **は恒星日（86164.0989 秒）**。基準のデータを暦日で作ると、黙って誤る（既知の問題 23）。hour の係数は 1.7.0 で 3600 に直した（既知の問題 3）

11 tacode

src/adapter_tacode/adapter_tacode.py。軌道解析コード Tacode を呼ぶ。

キー	既定	選べる値	説明・注意
tacode.code_system	必須	python3・fortran	Tacode の版。変数の名前・見出しの読み方・制御ファイルの書き換え方が変わる
tacode.work_dir・case_dir・step_digit	必須	文字列・文字列・整数	作業ディレクトリ（**起動時に消して作り直す**）。反復ごとのケースは <work_dir>/<case_dir><**番号**>（番号は step_digit 桁の 0 埋め）
tacode.directory_path_specify・manual_path	必須	manual	テンプレートのケースの場所
tacode.filename_control	必須	文字列	パラメータを書き込む制御ファイル（例 orbit.ctl）
tacode.filename_result	必須	文字列	Tacode の軌道の出力（例 trajectory.dat）
tacode.cmd_code	必須	実行ファイル	ケースのディレクトリの中で起動する
tacode.target_time	必須	数値（日。flag_with_timeerror なら秒）	Tacode の時刻に足す開始の時刻。基準のデータの中の開始点を探すのにも使う
tacode.time_start・time_end	必須	数値（同上）	誤差を評価する区間
tacode.result_dir	必須	文字列	ケースごとの出力の置き場
tacode.flag_output・filename_output	必須（名前は必須*）	True・False、文字列	ケースごとの軌道を書き出す。名前には _caseNNNN が入る
tacode.convert_timeunit_tapost	—	—	**読まれない**

基準のデータの時刻の範囲の外にある点は比べず、その数を警告する（1.7.0 から。以前は配列の末尾へ回り込んでいた。既知の問題 6）。警告が出たら、target_time・time_start・time_end と基準のデータの時刻を見直すこと。

12 simple_function

src/adapter_template/adapter_simple_function.py。外部コードの要らない、解析的な目的関数。目的関数は f_k(x) = Σ_i (x_i − 2(k − 1))^2（k = 1, …, num_objectives）で、ずらした球関数である。function_type: rastrigin では、同じ位置に最小 0 を持つ Rastrigin 関数 f_k(x) = Σ_i [z_i^2 − 10 cos(2 π z_i) + 10]、z_i = x_i − 2(k − 1) になる（整数の格子点の近くに局所解がある）。最大化の符号に対応し、スレッドからの同時の評価（PSO.num_parallel）にも対応している（テストと検証に使う）。

キー	既定	選べる値	説明・注意
simple_function.function_type	sphere	sphere・rastrigin	関数の種類
simple_function.flag_output	必須	True・False	関数の形を 1 次元で書き出す。**真のときは num_objectives が 2 以上でないと止まる**（2 列目を書くため）
simple_function.result_dir・filename_output・function_discrete・function_bound_min・function_bound_max	必須*（上が真）	文字列・文字列・整数・数値・数値	書き出しの場所と、標本点の数・範囲

13 example_externalcode

src/adapter_template/adapter_example_externalcode.py。外部コードを呼ぶアダプタの雛形。

キー	既定	選べる値	説明・注意
example_externalcode.work_dir・case_dir・step_digit	必須		tacode と同じ
example_externalcode.directory_path_specify・manual_path	必須	manual	テンプレートの場所
example_externalcode.filename_control・filename_result	必須	文字列	制御ファイル（YAML のリストの形で書き換える）と、結果（列 x・y）
example_externalcode.cmd_externalcode	必須	実行ファイル	**他のアダプタの cmd_code と名前が違う**
example_externalcode.x_min・x_max・x_offset	必須	数値	誤差を評価する x の区間と、計算の x に足すずれ
example_externalcode.result_dir・flag_output・filename_output	必須（名前は必須*）		ケースごとの出力

14 heatcond

src/adapter_heatcond/adapter_heatcond.py。熱伝導の解析コードを呼ぶ。

キー	既定	選べる値	説明・注意
heatcond.work_dir・case_dir・step_digit・directory_path_specify・manual_path・cmd_code	必須		tacode と同じ
heatcond.filename_control・filename_result	必須	文字列	制御ファイル（name を含む行の先頭から N 語を書き換える）と、履歴のファイル（Tecplot）
heatcond.x_min・x_max・x_offset	必須	数値 [s]	誤差を評価する時刻の区間と、Time[s] に足すずれ

キー	既定	選べる値	説明・注意
heatcond.result_dir・flag_output・filename_output	必須（名前は必須*）		ケースごとの出力
heatcond.variables_output	必須*（flag_output）	Time[s]・T[K]・Alt.[km]・Dens[kg/m3]・Tback[K]・Tatm[K]・Htcoef・Re・Solar[W/m2]・Heatflux[W/m2] の一部	書き出す列

15 cage

src/adapter_cage/adapter_cage.py。

キー	既定	選べる値	説明・注意
cage.work_dir・case_dir・step_digit・directory_path_specify・manual_path・cmd_code	必須		tacode と同じ
cage.filename_control・filename_result	必須	文字列	制御ファイル（YAML のリスト）と、カンマ区切りの結果
cage.var_x・var_y	必須	文字列・リスト	結果のファイルの x の列と、reference.var_y と順に比べる y の列
cage.x_min・x_max・x_offset	必須	数値	**読むが効かない**（区間を切る処理がコメントアウトされている）
cage.result_dir・flag_output・variables_output・filename_output	必須（後ろの 2 つは必須*）		ケースごとの出力

16 shapeoptimizer

src/adapter_shapeoptimizer/adapter_shapeoptimizer.py。SU2 と、preCICE で連成した形状の変形（src_interface/）を呼ぶ。目的関数の値は、num_objectives: 1 なら抗力、2 なら抗力とモーメントである（1.7.0 から。既知の問題 15）。

キー	既定	選べる値	説明・注意
shapeoptimizer.work_dir・case_dir・step_digit	必須		作業ディレクトリ。ケースは <case_dir>_rank<番号>
shapeoptimizer.directory_path_specify・manual_path	必須	manual	テンプレートの場所

キー	既定	選べる値	説明・注意
shapeoptimizer.cmd_code	必須	実行ファイル	ケースのディレクトリの中で起動する（例 ./run_SU2-OPT_pps.sh）。環境変数 BOREALIS_SLOT・BOREALIS_NUM_CPU・BOREALIS_CPU_SET を渡す。**計算が終わるまで戻らないこと**（末尾を wait & にしない）
shapeoptimizer.result_dir	必須	文字列	作るだけで、何も書かない
shapeoptimizer.in_sequential	必須	True・False	真なら粒子ごとに連成計算を常駐させ、ステップごとにファイルの名前を変えてやりとりする。PSO でだけ使える
shapeoptimizer.work_dir_series	必須	文字列	変位と力のファイルを置く、ケースの中のディレクトリ（例 optimization）
shapeoptimizer.filename_displacements・filename_forces	必須	文字列	Borealis が書く変位（マーカー × 次元）と、読む力（1 行目が力、2 行目がモーメント）。目的関数は **[力** [0]**, モーメント** [0]]
shapeoptimizer.step_digit_series	必須	整数	逐次モードの _stepNNNNN の桁
shapeoptimizer.flag_surfacearea_constraint	False	True・False	表面積の制約
shapeoptimizer.work_dir_surfacearea・filename_surfacearea	fluid・surfacearea_su2.dat	文字列	表面積のファイル（見出しと area, area_init）
shapeoptimizer.tolerance_surfacearea	0.05	数値	表面積の相対変化がこれ以上ならペナルティ
shapeoptimizer.penalty_surfacearea	必須*（制約を破ったとき）	数値	そのペナルティ
shapeoptimizer.num_cpu_per_case	2	整数	ケースあたりのコア数
shapeoptimizer.cpu_offset	0	整数	最初のコア。スロット s のコアは cpu_offset + s × num_cpu_per_case 番目から
shapeoptimizer.cpu_assignment	affinity	affinity・absolute	affinity はこのプロセスに許されたコアの並び（os.sched_getaffinity。ジョブスケジューラが割り当てたコア）から数える。absolute は 0 番からの絶対番号。足りなければ起動時に止まる
shapeoptimizer.flag_cpu_set	True	True・False	偽なら BOREALIS_CPU_SET を渡さない
shapeoptimizer.polling_interval	3.0	数値 [s]	結果のファイルを見る間隔。1 件が短いケースでは縮める
shapeoptimizer.timeout_finalize	600.0	数値 [s]	結果のあと、cmd_code の終了を待つ上限。超えたらプロセスグループを止める
shapeoptimizer.penalty_failure	1e10	数値	結果のファイルが得られなかったときに返す値（2 つの目的とも）
shapeoptimizer.power_variable_objectivefunction・filename_result・flag_output・filename_output、boundary[].coordinate	—	—	**読まれない**

同時に評価する数は <kind_optimizer>.num_parallel（PSO・GA・ABC のどれでも）で決める。Borealis は mpirun なしで起動する。

17 user

src/hidden_adapter_user/adapter_user.py(kind_adapter: user)。利用者が自分のアダプタを置く場所。キーは example_externalcode とほぼ同じで、user.work_dir・case_dir・step_digit・directory_path_specify・manual_path・filename_control・filename_result・cmd_code・result_dir・x_min・x_max・flag_output・filename_output を読む（x_offset とペナルティは無い）。**パラメータの範囲は Bayesian_optimization.boundary から読む。**

18 読まれないキーの一覧

雛形と testcase の yml に書いてあるが、コードが読まないキー（2026-10-19 に、すべての borealis.yml を調べた）。書いても効かない。1.7.0 からは、起動時に警告が出る。

キー	書いてある所	代わりに
PSO.tolerance	src/borealis.yml、testcase/work_cage・work_example_externalcode・work_heatcond	PSO.tolerance_residual
PSO.times_spte_pso	testcase/work_shapeoptimizer	無し
PSO.kind_residual_computation	testcase/work_shapeoptimizer（relative_value）	表示されるだけ
GA.mutation_limit・GA.tolerance	testcase/work_heatcond・work_simple_function	1.6 までの試作の GA のキー
Bayesian_optimization.initial_design_type・model_update_interval・acquisition_optimizer_type・verbosity・flag_image_acquisition	ベイズ最適化のセクションのある yml すべて	無し（GPyOpt の選択肢）
tacode.convert_timeunit_tapost	src/borealis.yml、tacode のケースすべて	無し
shapeoptimizer.power_variable_objectivefunction ほか	testcase/work_shapeoptimizer	無し（既知の問題 15）
reference.timeunit（heatcond のケース）	testcase/work_heatcond	tacode だけが読む
parameter_optimized.flag_penalty・penalty_value（simple_function のケース）	testcase/work_simple_function	simple_function はペナルティを使わない

逆に、必須なのに yml に無いもの（そのケースを走らせると KeyError で止まるはずのもの。実行はしていない）。

yml	足りないキー
src/borealis.yml の cage のセクション	cage.var_x・cage.var_y